OPCODA: ATLAS DA ARQUITECTURA

Mateus Costa de Oliveira Santos
Rafael Augusto Garcez Elisei
Prof. Me. Warner Brezolin (orientador)

1 Mapa do repositorio

O Opcoda e' um sintetizador granular que transforma ficheiros executaveis do formato Portable Executable em som. A thread de audio nunca le disco: a leitura, a analise e a conversao do material acontecem na thread de interface, e a unica passagem entre as duas e' uma fila circular sem trava que carrega ponteiro cru.

O repositorio tem duas camadas. `opcoda_core` e' C++20 puro e nao inclui JUCE em lado nenhum; `opcoda_plugin` e' a unica camada que depende do framework e da thread de audio do host. O nucleo existe para poder ser testado sem uma janela e sem um host.

Pasta	Papel
`src/opcoda\_core/pe`	Leitura defensiva do PE, conversao de bytes, selecao de regiao
`src/opcoda\_core/entropy`	Entropia de Shannon por janela
`src/opcoda\_core/dsp`	Motor granular, janelas, filtros, limitador
`src/opcoda\_core/rt`	Fila sem trava, transporte, notas, guard de alocacao
`src/opcoda\_core/view`	Rastro dos graos, sem JUCE
`src/opcoda\_plugin/source`	Processador JUCE, editor, grelha de bytes

2 Contexto

O Opcoda entra em cena por tres vias: o utilizador larga um executavel, o host entrega MIDI e parametros, e o host entrega o estado de um projecto guardado. Do outro lado, o plugin devolve audio, devolve estado e escreve ficheiros de preset.

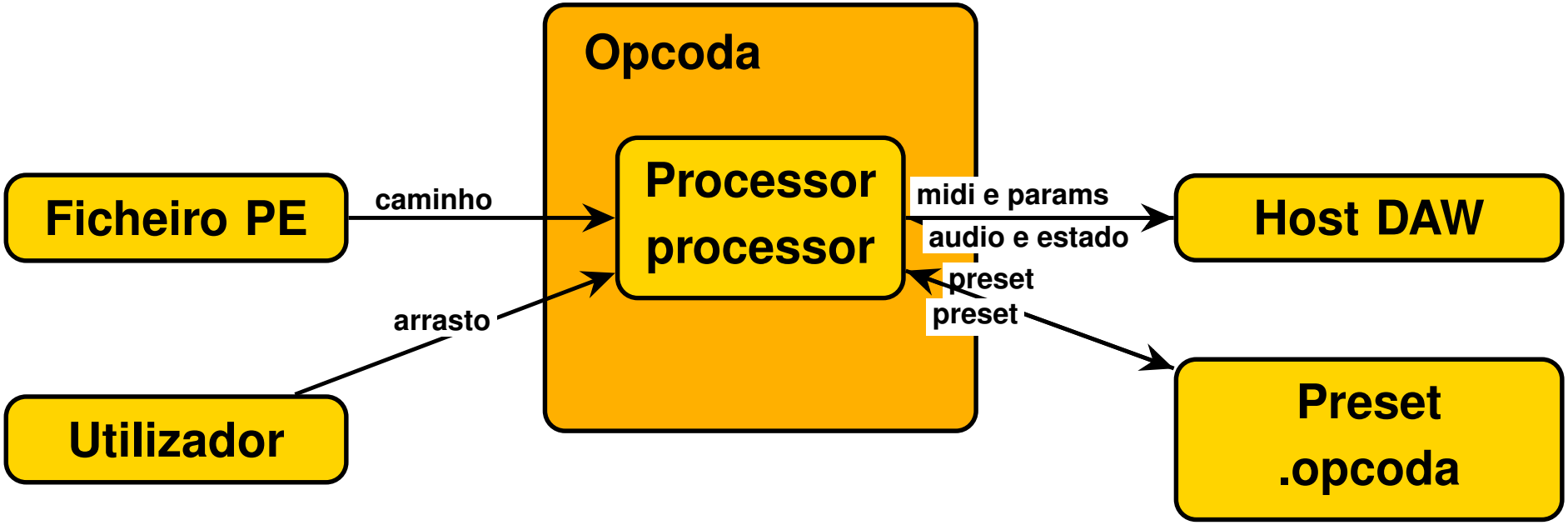


Figura 1. Contexto. O que entra no Opcoda vem da esquerda, o que sai vai para a direita, e o plugin fica no meio.

Fluxo	Origem	Destino	Tipo
caminho	`plugin\_processor.cpp:145`	`pe\_parser.cpp:65`	`vector<uint8_t>`
arrasto	`plugin\_editor.h:66`	`plugin\_processor.cpp:144`	`juce::String`
midi e params	`plugin\_processor.cpp:471`	`note\_tracker.cpp:7`	`MidiBuffer` e `float`
audio e estado	`plugin\_processor.cpp:499`	barramento do host	`float`, dois canais
preset	`plugin\_processor.cpp:658`	ficheiro `.opcoda`	texto XML

O ficheiro PE entra pela thread de interface e nunca pela thread de audio. O que viaja para o motor e' um ponteiro, e o que o motor devolve e' o ponteiro antigo, para que a interface possa libertar a memoria quando quiser.

3 Contentores

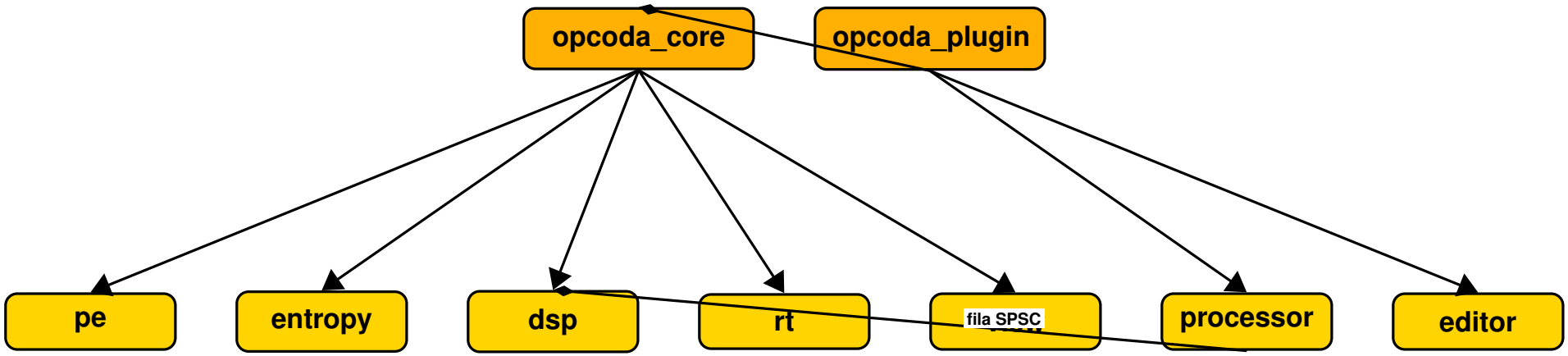


Figura 2. Contentores. O nucleo C++20 puro abaixo, o plugin JUCE acima, e a fila SPSC a unica passagem entre os dois.

Relacao	Onde	O que significa
`opcoda_core` para `pe`	`CMakeLists.txt:22`	leitura do PE, conversao de bytes, selecao de regiao
`opcoda_core` para `entropy`	`CMakeLists.txt:27`	entropia de Shannon por janela
`opcoda_core` para `dsp`	`CMakeLists.txt:33`	motor granular, janelas, filtros, limitador
`opcoda_core` para `rt`	`CMakeLists.txt:34`	fila sem trava, transporte, notas, guard de alocacao
`opcoda_core` para `view`	`grain\_trail.h:1`	rastro dos graos, sem JUCE
`opcoda_plugin` para `opcoda_core`	`opcoda\_plugin/CMakeLists.txt:82`	o plugin liga o nucleo
`opcoda_plugin` para `processor`	`opcoda\_plugin/CMakeLists.txt:61`	fronteira com o host
`opcoda_plugin` para `editor`	`opcoda\_plugin/CMakeLists.txt:63`	interface
`processor` para `dsp`	`plugin\_processor.cpp:356`	fila SPSC, sem alocacao

3.1 Onde esta a fronteira de tempo real

`PluginProcessor::processBlock` e' o unico ponto onde a thread de audio entra. A `PluginProcessor` tem duas filas: `incoming_` leva material novo para o motor, `returned_` traz de volta o ponteiro que o motor ja nao usa. As duas tem capacidade 4 e `SpscRing` exige tipo trivial, por isso o que circula e' ponteiro cru e nao `shared_ptr`.

O buffer vive em `owned_`, que e' da thread de interface. A interface liberta-o em `PluginEditor::refresh`, a 60 Hz, e so depois de o motor o ter devolvido.

4 Fluxo de dados

4.1 Nivel 0

Da esquerda para a direita: o ficheiro entra, o audio sai. As duas caixas cinzentas sao as duas threads, e a fila SPSC e' a unica passagem entre elas.

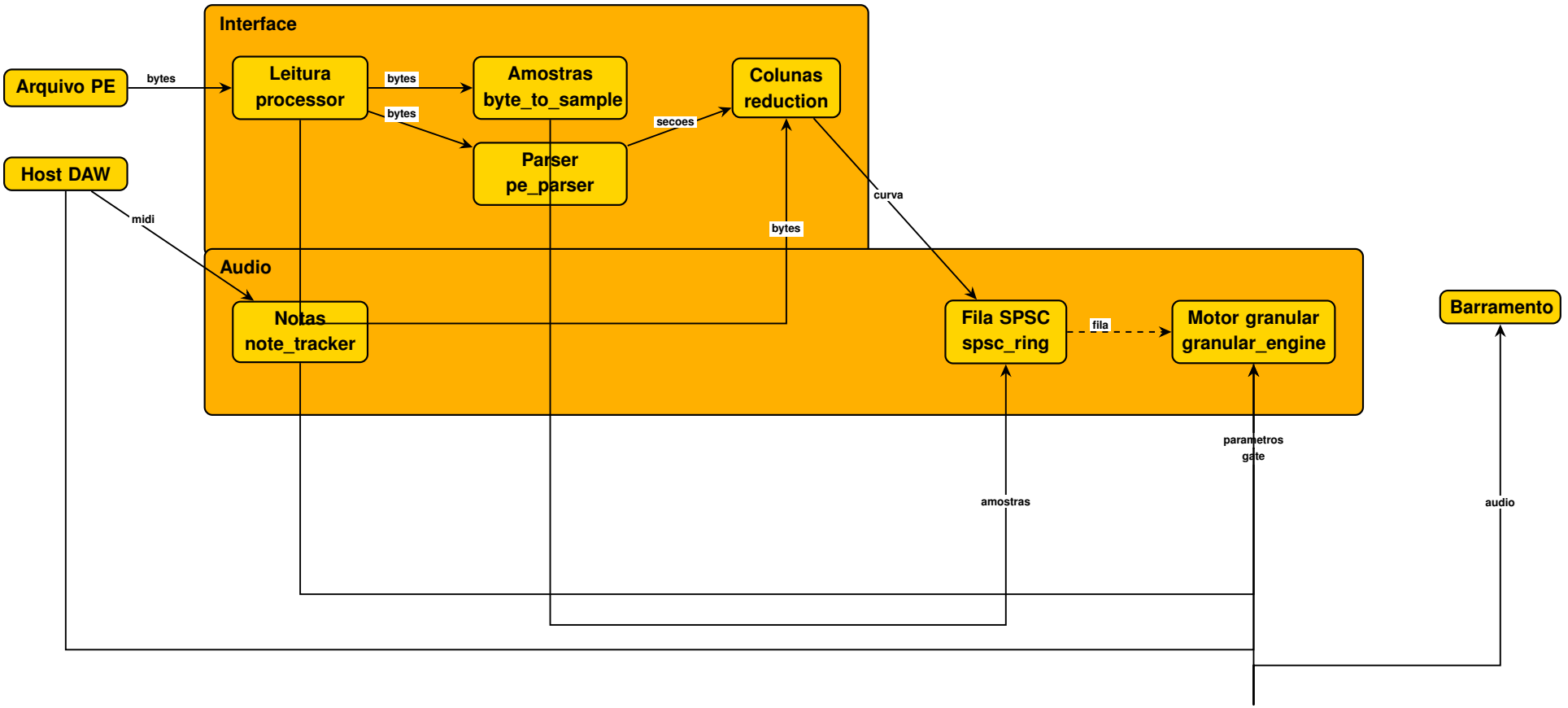


Figura 3. Fluxo de dados de nivel 0. Da leitura do ficheiro aos samples no barramento, com a fila SPSC a marcar a fronteira entre as duas threads.

Fluxo	Tipo	Origem	Destino
bytes	`vector<uint8_t>`	`plugin\_processor.cpp:145`	`pe\_parser.cpp:65`
bytes	`vector<uint8_t>`	`plugin\_processor.cpp:145`	`byte\_to\_sample.cpp:5`
bytes	`vector<uint8_t>`	`plugin\_processor.cpp:145`	`column\_reduction.cpp:54`
secoes	`pe::Section`	`pe\_parser.cpp:126`	`column\_reduction.cpp:54`
amostras	`vector<float>`	`byte\_to\_sample.cpp:14`	`plugin\_processor.cpp:352`
curva	`vector<float>`	`column\_reduction.cpp:163`	`plugin\_processor.cpp:353`
fila	`PublishedMaterial`	`plugin\_processor.cpp:356`	`granular\_engine.cpp:118`
midi	`MidiBuffer`	`plugin\_processor.cpp:471`	`note\_tracker.cpp:7`
gate	`bool`	`note\_tracker.cpp:48`	`granular\_engine.cpp:65`
parametros	`GranularParams`	`plugin\_processor.cpp:484`	`granular\_engine.cpp:280`
audio	`float`, dois canais	`granular\_engine.cpp:478`	`plugin\_processor.cpp:499`

A curva de entropia viaja com as amostras porque o motor nao a pode calcular. Medir a entropia de toda a regiao dentro de `processBlock` seria trabalho do tamanho da regiao dentro da thread de audio.

4.2 Nivel 1: o ingest

Do ficheiro ate a fila. Tudo o que acontece aqui corre na thread de interface, e por isso pode alocar.

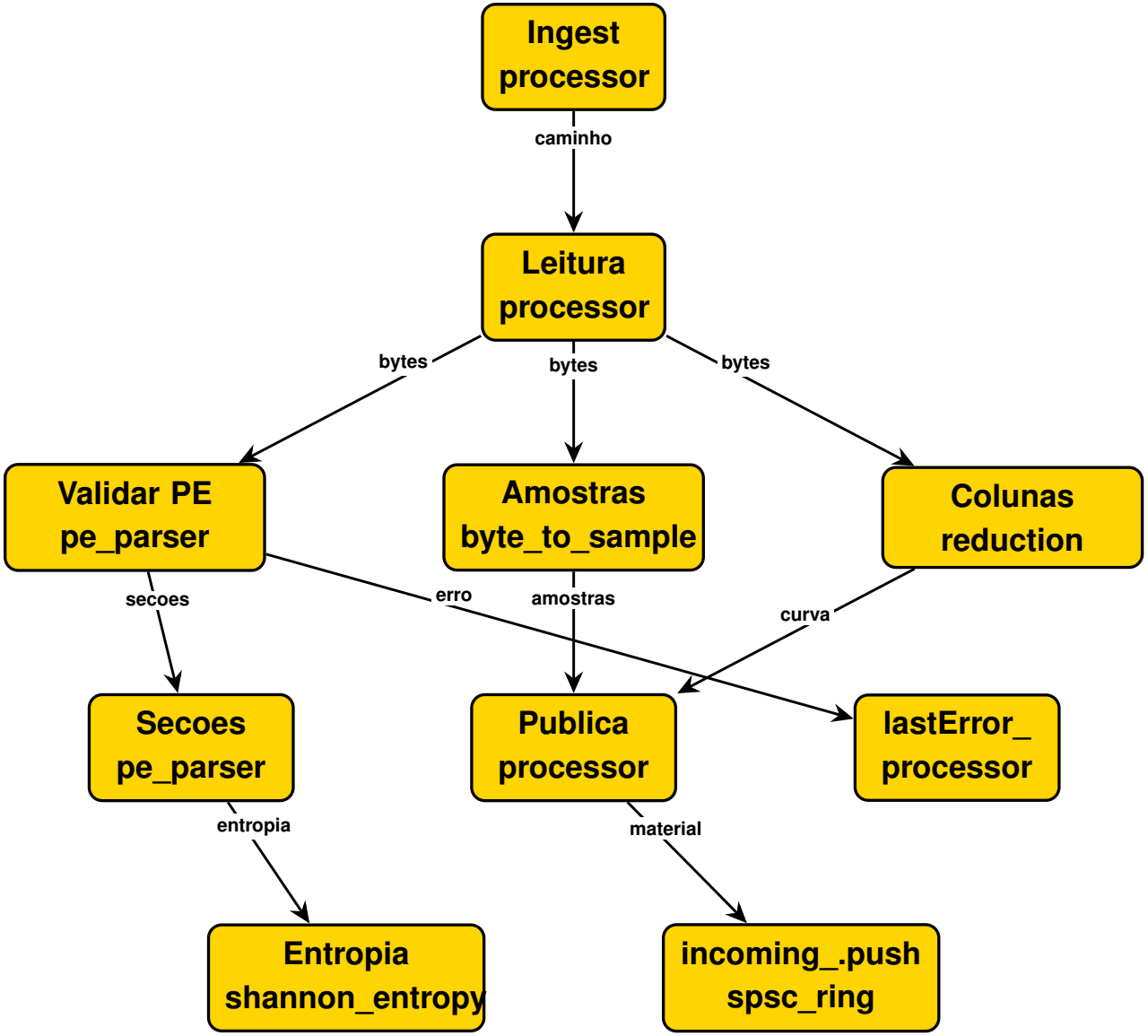


Figura 4. Nivel 1 do ingest. Do ficheiro ate a fila, com o erro tipado a sair pelo ramo de baixo.

Passo	Codigo	O que move
1	`plugin\_processor.cpp:144`	o caminho do ficheiro
2	`plugin\_processor.cpp:160`	os bytes crus
3	`pe\_parser.cpp:65`	a estrutura do PE, ou um erro tipado
4	`pe\_parser.cpp:126`	uma `Section` por seccao
5	`shannon\_entropy.cpp:21`	os bits por byte de cada seccao
6	`byte\_to\_sample.cpp:14`	as amostras em $[-1, 1]$
7	`column\_reduction.cpp:163`	minimo, maximo, RMS e entropia de cada coluna
8	`plugin\_processor.cpp:317`	as amostras e a curva, em dois vectores
9	`plugin\_processor.cpp:356`	os dois ponteiros, para a fila
10	`plugin\_processor.cpp:167`	o erro, como `const char*`

4.3 Nivel 1: o bloco de audio

Do host ate ao barramento. Nada disto aloca, nada disto bloqueia, e nada disto le disco.

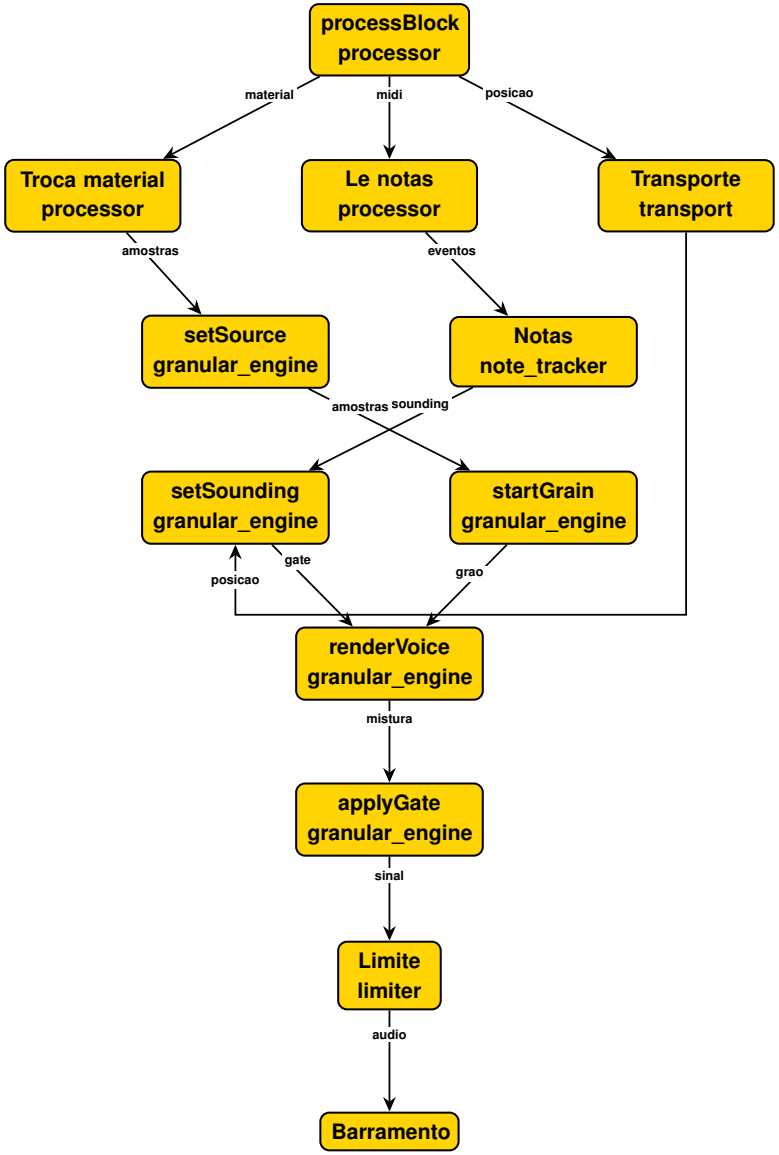


Figura 5. Nivel 1 do bloco de audio. Da leitura do MIDI ate ao limitador, tudo na thread de audio.

Passo	Codigo	O que move
1	`plugin\_processor.cpp:465`	o buffer e o MIDI do bloco
2	`plugin\_processor.cpp:356`	o `PublishedMaterial` que chegou
3	`plugin\_processor.cpp:387`	o ponteiro das amostras e o da curva
4	`plugin\_processor.cpp:414`	os eventos MIDI do bloco
5	`note\_tracker.cpp:7`	as notas premidas e o pedal
6	`transport.cpp:122`	a fracao da regiao que o transporte leu
7	`granular\_engine.cpp:65`	o comando de abrir ou fechar o gate
8	`granular\_engine.cpp:178`	o grao: posicao, ganho, pan e janela
9	`granular\_engine.cpp:392`	a amostra interpolada, ja filtrada
10	`granular\_engine.cpp:495`	o nivel do envelope, aplicado a mistura
11	`limiter.cpp:62`	o sinal saturado
12	`plugin\_processor.cpp:499`	os dois canais no barramento

O gate vem antes do limitador de proposito. Fechar a rampa antes de saturar mantem o tecto de saida independente de quantas notas estao a soar.

5 Entradas e saidas

5.1 Entradas

Entrada	Sitio	Dado
dialogo de ficheiro	`plugin\_editor.cpp:1430`	caminho, `juce::String`
arrasto de ficheiro	`plugin\_editor.h:66`	`juce::StringArray`
estado do host	`plugin\_processor.cpp:637`	bloco XML
MIDI do host	`plugin\_processor.cpp:471`	`juce::MidiBuffer`
parametros do host	`plugin\_processor.cpp:83`	`float`, 28 parametros
taxa de amostragem	`plugin\_processor.cpp:120`	`double`
fila de entrada	`plugin\_processor.cpp:356`	`PublishedMaterial`
fila de devolucao	`plugin\_processor.cpp:380`	`PublishedMaterial`
preset do disco	`plugin\_processor.cpp:601`	`juce::XmlElement`
ficheiro PE pelo nucleo	`pe\_parser.cpp:166`	`vector<uint8_t>`

5.2 Saidas

Saida	Sitio	Dado
audio para o host	`plugin\_processor.cpp:499`	`float`, dois canais
estado para o host	`plugin\_processor.cpp:595`	bloco XML
parametro para o host	`plugin\_processor.cpp:457`	`float` normalizado
preset para o disco	`plugin\_processor.cpp:658`	texto XML
telemetria para a interface	`plugin\_processor.cpp:522`	atomicos
grafos para a interface	`granular\_engine.cpp:492`	`GrainTelemetry`
erro de parser	`plugin\_processor.cpp:167`	`const char*`
erro de abertura	`plugin\_processor.cpp:147`	`const char*`
erro de fila cheia	`plugin\_processor.cpp:359`	`const char*`
erro de fonte em falta	`plugin\_processor.cpp:632`	`const char*`

5.3 Erros

Os erros do Opcoda sao codigos de texto, nao excepcoes. `pe::toString` traduz o enumerado `PeError` para uma cadeia estavel, e a interface mostra a cadeia tal e qual.

Codigo	Quando
`E_OPEN`	o ficheiro nao abre
`E_EMPTY`	o ficheiro esta vazio
`E_TRUNCATED`	a leitura devolveu menos bytes do que o tamanho declarado
`E_BAD_MZ`	a assinatura `MZ` nao esta no inicio
`E_BAD_E_LFANEW`	o offset do cabecalho PE esta fora do ficheiro
`E_BAD_PE`	a assinatura `PE` nao esta no sitio
`E_BAD_OPTIONAL_HEADER`	o cabecalho opcional e' curto demais
`E_TOO_MANY_SECTIONS`	o numero de seccoes e' zero ou passa de 96
`E_BAD_SECTION_TABLE`	a tabela de seccoes nao cabe no ficheiro
`E_OOB`	uma seccao declara mais bytes do que o ficheiro tem
`E_BUSY`	a fila de entrada esta cheia
`E_SOURCE_MISSING`	o projecto aponta para um ficheiro que ja nao esta la

5.4 Caminho quente

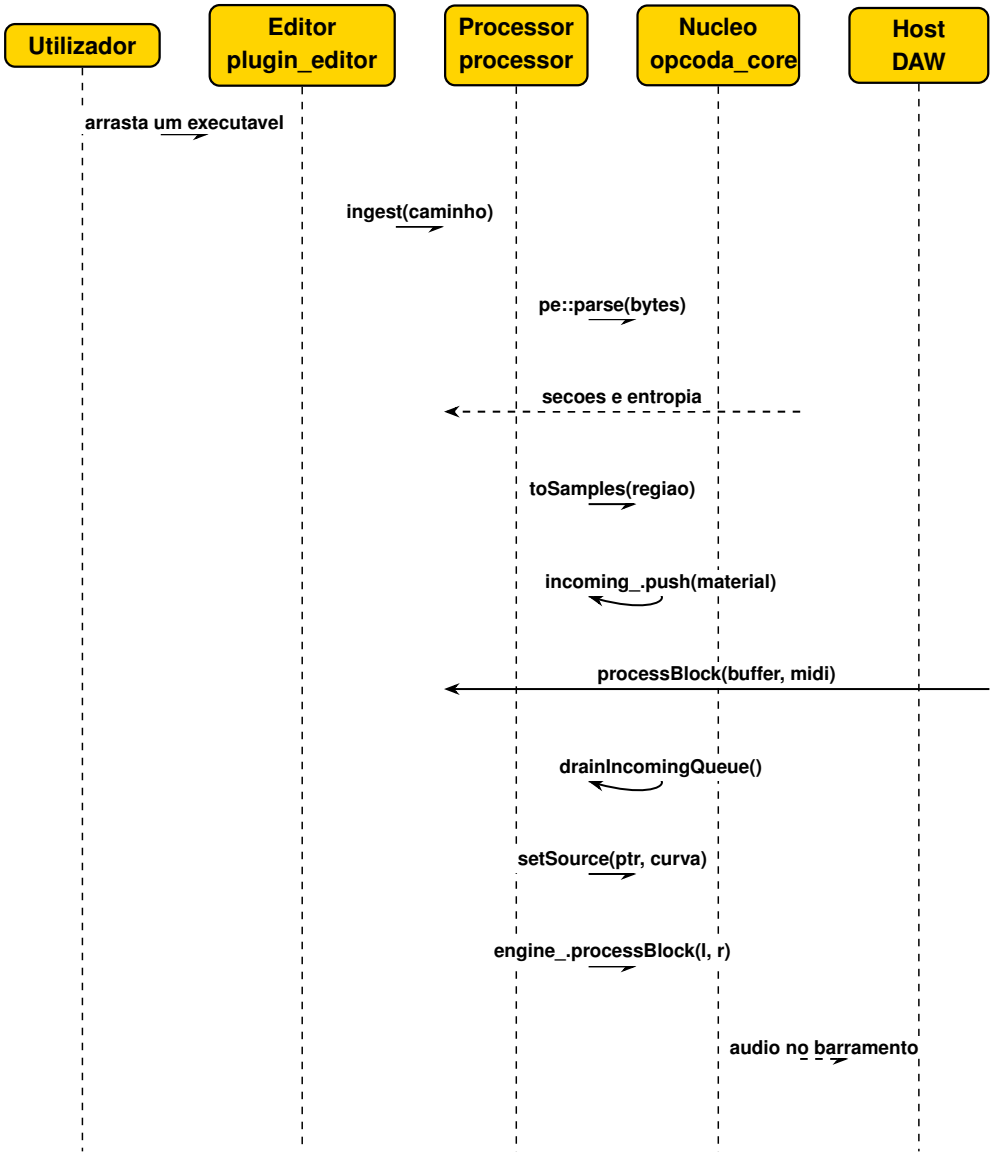


Figura 6. Caminho quente. A ordem das chamadas desde o arrasto do ficheiro ate ao audio no barramento.

O arrasto e' tratado em `PluginEditor::filesDropped`, que chama `ingestFirst`. O dialogo de ficheiro faz o mesmo por `chooseFile`. Os dois caminhos acabam no mesmo `PluginProcessor::ingest`, e nao ha caminho paralelo.

A troca de material acontece no inicio do bloco, em `drainIncomingQueue`. O motor recebe o ponteiro novo em `setSource` e o ponteiro antigo volta pela fila de devolucao. A interface liberta-o no `refresh` seguinte, a 60 Hz.